# StockFlow — Mini ERP de gestion de stock

Guide d'installation, de lancement et de vérification

StockFlow est une application web de démonstration pour gérer un catalogue de produits, les catégories, les clients, les commandes et les mouvements de stock. Ce dépôt contient le code source du frontend, de l'API et les scripts de création et d'initialisation de la base de données.

## Fonctionnalités

- Connexion avec authentification JWT et rôles `admin` / `staff`.
- Tableau de bord avec indicateurs et produits les plus vendus.
- Gestion des produits : création, consultation, modification, suppression réservée à l'administrateur, recherche par nom ou SKU et pagination.
- Gestion des catégories et des clients, avec modification des coordonnées d'un client.
- Création de commandes, ajout de lignes, calcul du total et suivi des statuts.
- Gestion des entrées et sorties de stock avec historique des mouvements.
- Vérification du stock lors de la confirmation d'une commande et refus des sorties qui rendraient le stock négatif.

## Technologies

- Frontend : React, Vite, JavaScript et CSS.
- API : Node.js, Express, JWT, bcryptjs et Zod.
- Base de données : PostgreSQL, avec requêtes SQL paramétrées.
- Exécution locale de PostgreSQL : Docker Compose.

## Prérequis

- Windows, macOS ou Linux.
- Node.js 20 ou une version ultérieure et npm.
- Docker Desktop démarré, avec le moteur Linux/WSL2 disponible, ou une instance PostgreSQL 16 accessible.

## Démarrage avec Docker (Windows PowerShell)

Ouvrir un terminal à la racine du projet, dans le dossier `stockflow`.

### 1. Démarrer PostgreSQL

```
docker compose up -d postgres
docker compose ps
```

Attendre que le conteneur `stockflow-db` soit indiqué comme prêt (`healthy`). Docker expose PostgreSQL sur `localhost:5432`, avec la base `stockflow` et les identifiants locaux définis dans `docker-compose.yml`.

### 2. Configurer et initialiser l'API

Dans un nouveau terminal :

```
cd server
Copy-Item .env.example .env
```

Ouvrir `server/.env` et remplacer `JWT_SECRET` par une chaîne longue et aléatoire. Pour l'environnement Docker local, conserver `DATABASE_URL` et `CLIENT_ORIGIN` tels qu'ils figurent dans `.env.example`.

Puis installer les dépendances, créer le schéma, charger les données de démonstration et démarrer l'API :

```
npm install
npm run db:init
npm run db:seed
npm run dev
```

L'API écoute par défaut sur `http://localhost:4000`. L'endpoint de contrôle est `http://localhost:4000/api/health`.

Le script `npm run db:seed` réinitialise les données des tables du schéma StockFlow avant de charger les données de démonstration. Ne l'exécutez pas sur une base contenant des données à conserver.

### 3. Démarrer le frontend

Dans un autre terminal, depuis la racine du projet :

```
cd client
npm install
npm run dev
```

Ouvrir l'adresse affichée par Vite, par défaut `http://localhost:5173`.

### Comptes de démonstration

Mot de passe commun : `Stockflow123!`

- Administrateur : `admin@stockflow.local`
- Personnel : `amine@stockflow.local`
- Personnel : `sara@stockflow.local`

Ces comptes sont réservés à la démonstration locale. Remplacez ou supprimez-les avant tout déploiement.

## Configuration

Variables dans `server/.env` :

| Variable | Usage | Valeur locale par défaut |
|---|---|---|
| `PORT` | Port de l'API | `4000` |
| `DATABASE_URL` | Connexion PostgreSQL | `postgresql://postgres:postgres@localhost:5432/stockflow` |
| `JWT_SECRET` | Signature des jetons de connexion | À remplacer par une valeur aléatoire |
| `CLIENT_ORIGIN` | Origine autorisée par CORS | `http://localhost:5173` |

Le frontend utilise `http://localhost:4000/api` par défaut. Pour changer cette adresse, définir `VITE_API_URL` avant de lancer ou construire le client.

## Vérification de la base de données

Dans pgAdmin ou un client SQL, sélectionner la base `stockflow`, puis exécuter :

```
SELECT COUNT(*) FROM products;
SELECT COUNT(*) FROM categories;
SELECT COUNT(*) FROM clients;
SELECT COUNT(*) FROM orders;
SELECT id, name, sku, stock_quantity FROM products ORDER BY id;
```

Le jeu de démonstration contient 5 catégories, 15 produits, 5 clients, 10 commandes et les utilisateurs/mouvements associés.

## API

Toutes les routes métier (sauf la connexion) nécessitent un jeton transmis comme `Authorization: Bearer <token>`.

- Santé : `GET /api/health`
- Authentification : `POST /api/auth/login`, `GET /api/auth/me`
- Tableau de bord : `GET /api/dashboard`
- Catégories : `GET/POST /api/categories`, `PUT/DELETE /api/categories/:id`
- Produits : `GET /api/products?search=&page=&limit=`, `GET /api/products/:id`, `POST/PUT/DELETE /api/products/:id`
- Clients : `GET/POST /api/clients`, `PUT/DELETE /api/clients/:id`
- Commandes : `GET/POST /api/orders`, `POST /api/orders/:id/items`, `PATCH /api/orders/:id/status`
- Stock : `GET/POST /api/stock-movements?productId=`

Les rôles `admin` et `staff` peuvent gérer les opérations autorisées par l'application. La suppression des produits et les modifications des catégories sont réservées à `admin`.

## Modèle de données

Les tables principales sont `users`, `categories`, `products`, `clients`, `orders`, `order_items` et `stock_movements`. Les relations et contraintes sont documentées dans `docs/ERD.md` et définies dans `server/db/schema.sql`.

## Vérification de compilation

Pour vérifier la compilation de production du frontend :

```
cd client
npm run build
```

## Arrêter l'environnement local

Arrêter l'API et Vite avec `Ctrl+C`, puis, à la racine du projet :

```
docker compose down
```

Les données PostgreSQL restent dans le volume Docker. Pour les conserver, ne supprimez pas ce volume.

## Sécurité et limites de démonstration

Les mots de passe sont hachés avec bcryptjs, les routes vérifient le jeton et le rôle, les entrées API sont validées et les requêtes SQL sont paramétrées. Le dépôt ne doit pas contenir de fichier `.env` réel. Pour une mise en production, prévoir notamment un secret fort géré hors du dépôt, HTTPS, des sauvegardes, une gestion des comptes adaptée et une revue de sécurité.

## Documents de livraison

- Rapport technique illustré : `docs/Rapport-technique-StockFlow.pdf` et sa version Markdown `docs/RAPPORT-TECHNIQUE.md`.
- Schéma de la base de données au format PDF : `docs/ERD.pdf`, avec sa source éditable `docs/ERD.md`.
- Copie PDF de ce guide : `README.pdf`. Le fichier `README.md` reste le guide principal à la racine du projet.